

Specyfikacja Wymagań Oprogramowania

Projekt: System Ewidencji Wyjść Uczniów (Student Exit Tracker)

Wersja: 1.0

Data: 1 października 2026 r.

Autorzy: Bartosz Liszka, Wojciech Natora

1. Wstęp

1.1. Cel dokumentu

Celem niniejszego dokumentu jest przedstawienie pełnej specyfikacji wymagań funkcjonalnych i niefunkcjonalnych dla aplikacji desktopowej służącej do zarządzania i ewidencjonowania wyjść uczniów ze szkoły.

1.2. Zakres systemu

Aplikacja umożliwia:

- Prowadzenie bazy danych uczniów, klas oraz użytkowników systemu (nauczyciele, administratorzy).
- Szybkie rejestrowanie wyjść/powrotów uczniów w trakcie zajęć.
- Filtrowanie i podgląd uczniów w podziale na klasy.
- Bezpieczne zarządzanie uprawnieniami i kontami (hashowanie haseł).

1.3. Słownik pojęć

- **SRS** – *Software Requirements Specification* (Specyfikacja Wymagań Oprogramowania).
- **MVVM** – *Model-View-ViewModel* (Wzorzec architektoniczny warstwy prezentacji).
- **EF Core** – *Entity Framework Core* (Mapowanie obiektowo-relacyjne ORM).
- **FK / PK** – *Foreign Key / Primary Key* (Klucz obcy / Klucz główny w bazie danych).

2. Ogólny opis

2.1. Perspektywa produktu

Aplikacja jest samodzielnym programem desktopowym (Desktop Standalone) przeznaczonym na systemy Windows, działającym w oparciu o lokalną lub sieciową bazę danych SQLite.

2.2. Klasy użytkowników i uprawnienia

1. **Nauczyciel:** Podgląd historii wyjść swoich uczniów, rejestracja wyjść i powrotów uczniów, podgląd listy klas
2. **Administrator:** Pełen dostęp – dodawanie/edycja użytkowników, edycja klas i uczniów.

2.3. Środowisko uruchomieniowe

- **System operacyjny:** Windows 10 / 11 (64-bit)
- **Środowisko uruchomieniowe:** .NET 8.0 lub nowszy

3. Wymagania Funkcjonalne

Wymagania funkcjonalne definiują, co system musi robić.

3.1. Moduł Autoryzacji i Użytkowników (UC-01)

- **RF-01:** System musi umożliwiać logowanie za pomocą loginu i hasła.
- **RF-02:** Hasła w bazie danych muszą być szyfrowane algorytmem BCrypt.
- **RF-03:** Administrator musi mieć możliwość dodawania, edycji oraz usuwania kont użytkowników za pomocą dedykowanego dialogu edycji.

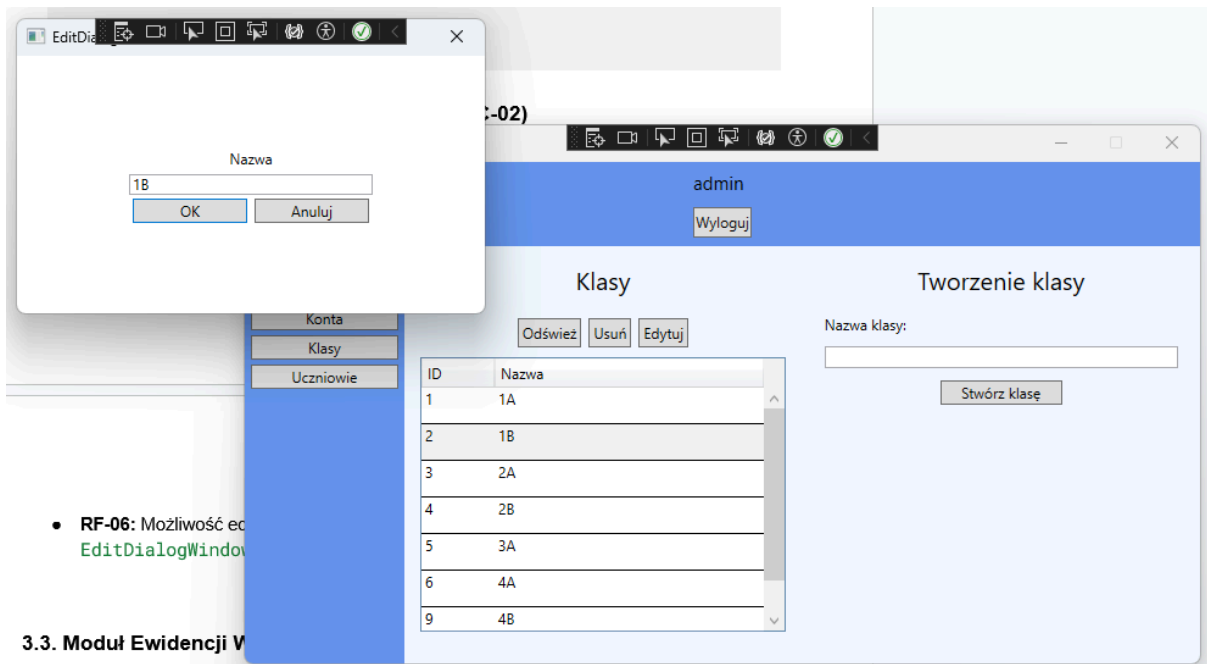


The image shows a login interface with a light gray background. At the top center, the word "Witaj" is displayed in a dark blue font. Below it, there are two white input fields with thin gray borders. The first field is labeled "Login:" and the second is labeled "Hasło:". Below these fields is a gray button with the text "Zaloguj się" in white.

3.2. Moduł Zarządzania Klasami i Uczniami (UC-02)

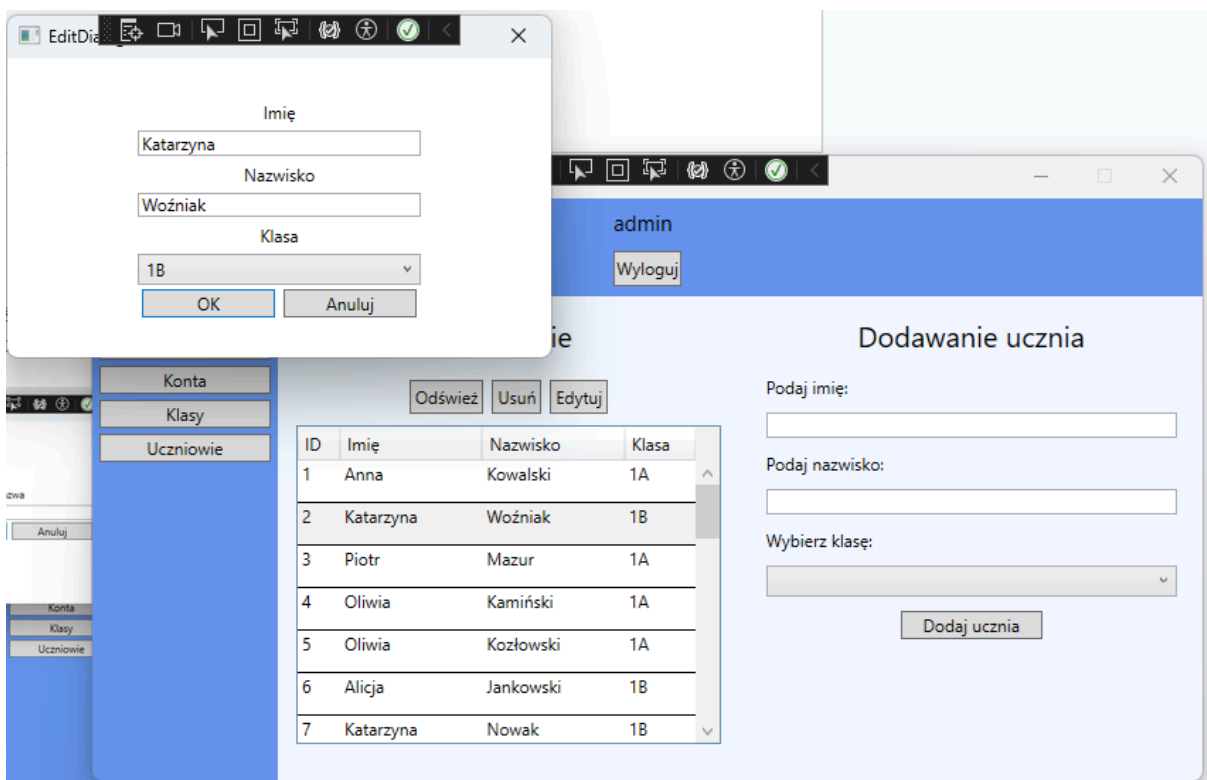
- **RF-04:** System musi umożliwiać filtrowanie uczniów po wybranej klasie z listy rozwijanej (ComboBox).
- **RF-05:** Wyświetlanie listy uczniów w tabeli (DataGrid) z uwzględnieniem nazwy klasy, statusu (w klasie / poza klasą) oraz zagnieżdżonych właściwości.

- **RF-06:** Możliwość edycji danych ucznia (Imię, Nazwisko, Klasa) w modalnym oknie `EditDialogWindow`.



- **RF-06:** Możliwość edycji danych ucznia (Imię, Nazwisko, Klasa) w modalnym oknie `EditDialogWindow`.

3.3. Moduł Ewidencji V



3.3. Moduł Ewidencji Wyjść (UC-03)

- **RF-07:** Zmiana statusu obecności ucznia (`IsInClass`) po kliknięciu przycisku akcji.

- **RF-08:** Automatyczne zapisywanie zdarzenia wyjścia z zebraniem daty, godziny oraz identyfikatora zarejestrowanego użytkownika.

4. Wymagania Niefunkcjonalne

4.1. Architektura i Jakość Kodu

- **RNF-01 (Wzorzec MVVM):** Warstwa widoku (**Views**) nie może zawierać logiki biznesowej w code-behind (**.xaml.cs**) – cała obsługa zdarzeń musi opierać się na **ICommand** / **RelayCommand** oraz wiązaniu danych (**DataBinding**).

```
Odwołania: 26
public class RelayCommand : ICommand
{
    private readonly Action _execute;
    Odwołania: 25
    public RelayCommand(Action execute) => _execute = execute;

    Odwołania: 0
    public bool CanExecute(object parameter) => true;
    Odwołania: 0
    public void Execute(object parameter) => _execute();
    public event EventHandler CanExecuteChanged;
}
```

- **RNF-02 (Polimorficzny System Dialogów):** Okna edycji muszą wykorzystywać uniwersalne okno **EditDialog** powiązane z **DialogViewModelBase** oraz dynamicznymi szablonami **DataTemplate**.

```
Odwołania: 5
public partial class EditDialog : Window
{
    Odwołania: 3
    public EditDialog()
    {
        InitializeComponent();

        DataContextChanged += (s, e) =>
        {
            if (DataContext is DialogViewModelBase vm)
            {
                vm.PropertyChanged += (sender, args) =>
                {
                    if (args.PropertyName == nameof(vm.DialogResult) && vm.DialogResult.HasValue)
                    {
                        DialogResult = vm.DialogResult;
                        Close();
                    }
                };
            }
        };
    }
}
```

4.2. Wydajność i Baza Danych (Performance)

- **RNF-03:** Czas odpowiedzi interfejsu przy przełączaniu klas nie może przekraczać 200 ms.
- **RNF-04:** Relacyjne pobieranie danych musi zapobiegać problemowi *Lazy Loading* poprzez stosowanie jawnego ładowania `.Include()` w EF Core.

```
Odwołania: 5
public void LoadStudents()
{
    using var db = new AppDbContext();
    var studentsFromDb = db.Students.Include(s => s.SchoolClass).ToList();
    Students.Clear();
    foreach (var user in studentsFromDb)
    {
        Students.Add(user);
    }
}
```

4.3. Usability i Interfejs Użytkownika

- **RNF-05:** Tabela `DataGrid` musi posiadać płynne przewijanie pionowe (`ScrollView.VerticalScrollBarVisibility="Auto"`) zapobiegające rozciąganiu układu.

Uczniowie

ID	Imię	Nazwisko	Klasa
1	Anna	Kowalski	1A
2	Katarzyna	Woźniak	1B
3	Piotr	Mazur	1A
4	Oliwia	Kamiński	1A
5	Oliwia	Kozłowski	1A
6	Alicja	Jankowski	1B
7	Katarzyna	Nowak	1B

- **RNF-06:** Wszystkie formularze edycyjne muszą umożliwiać zatwierdzenie klawiszem `Enter` oraz anulowanie zmian klawiszem `Esc`.

5. Baza Danych i Model Danych

5.1. Schemat Encji (ERD)

